

Artificial Intelligence (AI)

Lab Sheet No: 8

Introduction

According to Herbert Simon 1983, Learning denotes changes in the system that are adaptive in the sense that they enable the system to do the same task or tasks drawn from the same population more efficiently and more effectively next time.

“Evolved from the study of pattern recognition and computational learning theory in artificial intelligence, machine learning explores the study and construction of algorithms that can learn from and make predictions on data – such algorithms overcome following strictly static program instructions by making data-driven predictions or decisions, through building a model from sample inputs.” Arthur Samuel (pioneer in machine learning at IBM in 1959)

A useful model is built through a sequence of connected steps: preparing data, choosing a model, training it, checking whether it generalizes, and finally evaluating it on unseen data. This sequence is commonly called a machine learning pipeline.

Discovery (phase 1)

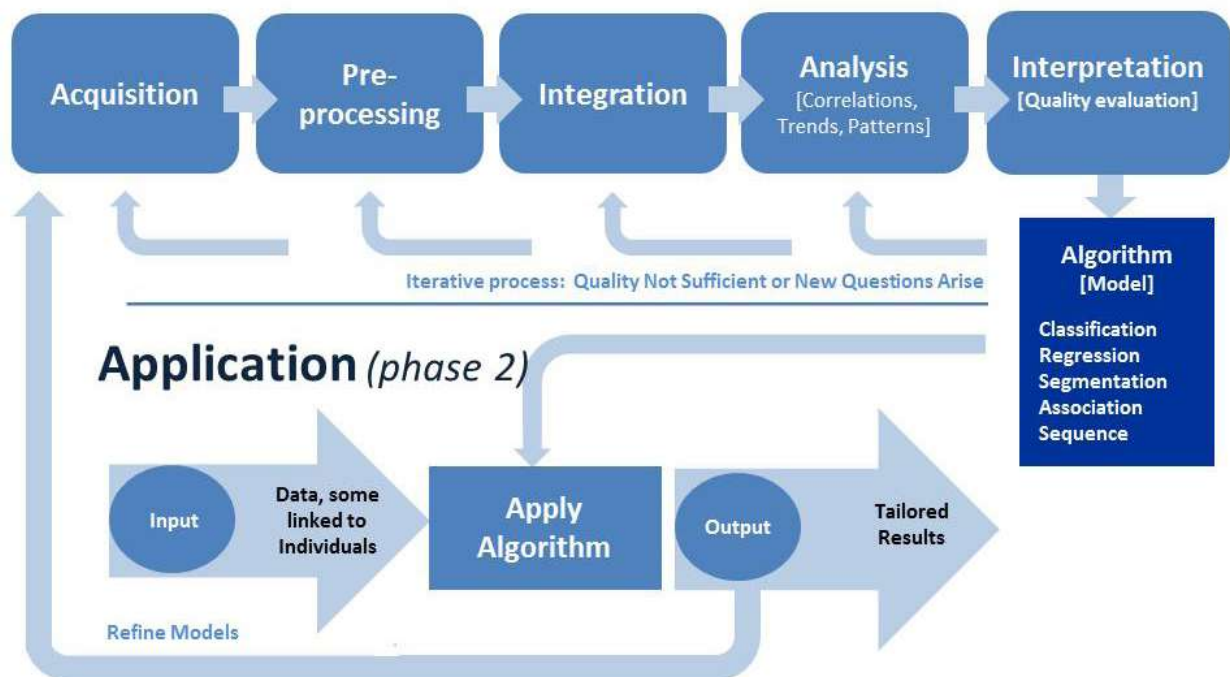


Figure 1: A simple machine learning pipeline

Generative and Discriminative Models

A generative model learns how the data for each class could have been produced. It models the underlying probability distribution and can use that information to classify new examples. Gaussian Naive Bayes is a common example.

A discriminative model focuses directly on separating classes or estimating the probability of a class given the input. Logistic Regression is a common example. In this lab, both models will be trained on the same dataset so that their behaviour can be compared fairly.

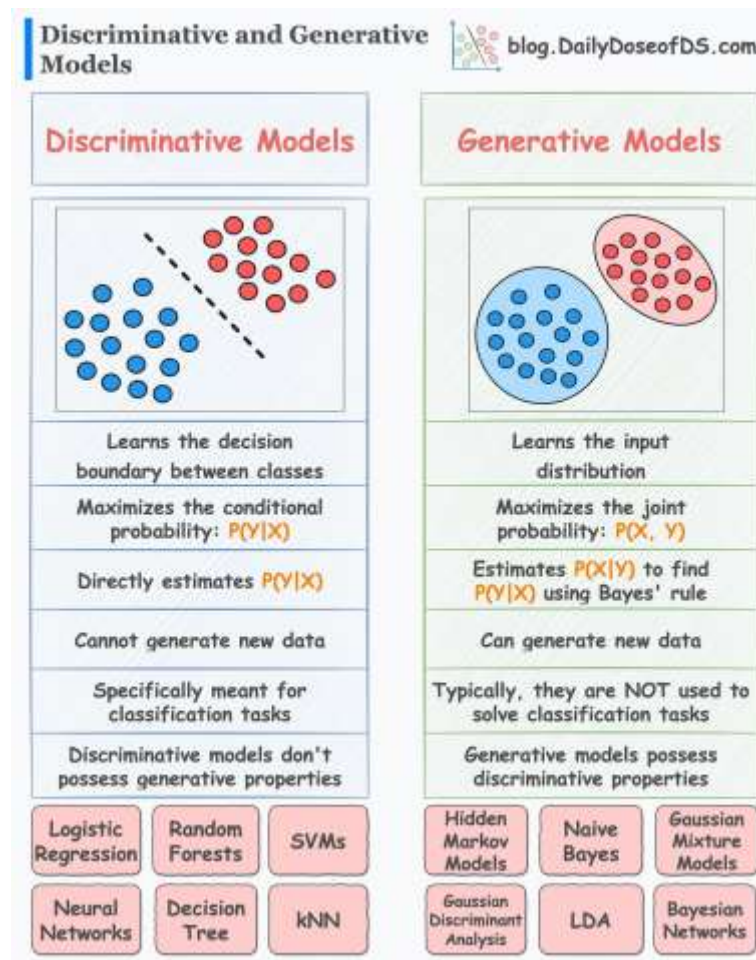


Figure 2: Generative and Discriminative Models

Learning Algorithm

A learning algorithm is the procedure used to find model parameters from data. During training, the model makes predictions, measures error using a loss function, and adjusts its parameters to reduce that error. Different algorithms use different procedures, but the goal is the same: learn a pattern that also works on unseen data.

Model Capacity, Underfitting and Overfitting

Model capacity describes how flexible a model is. A model with too little capacity may miss important patterns (underfitting). A model with too much capacity may memorize the training data and perform poorly on new data (overfitting). A useful model lies between these two extremes.

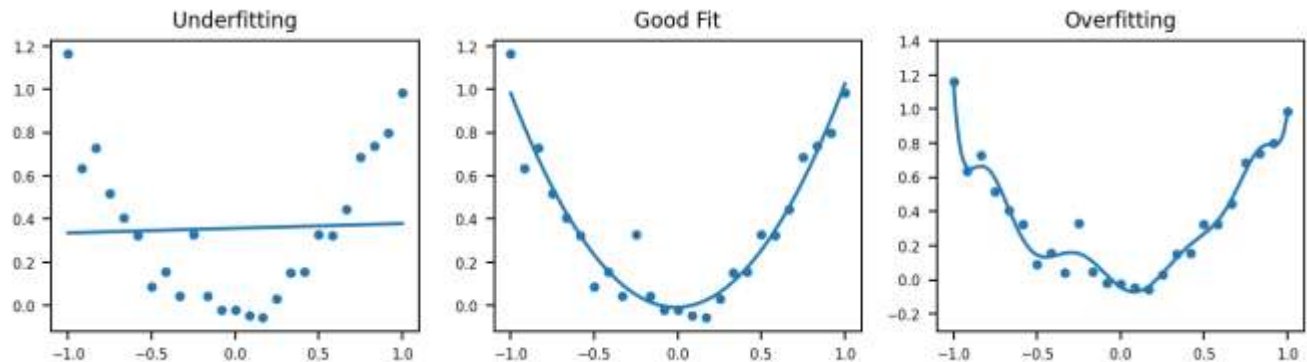


Figure 3: Conceptual view of underfitting, a good fit, and overfitting

Hyperparameters and Validation Sets

Parameters are learned from data, while hyperparameters are choices made by the user before or during training. Examples include the maximum depth of a decision tree, the value of k in k -nearest neighbours, or the strength of regularization in logistic regression.

To choose hyperparameters without using the final test set, the available data is commonly divided into training, validation, and test sets. The training set is used to learn model parameters, the validation set is used to choose hyperparameters or compare models, and the test set is kept aside for final evaluation.

Estimators, Bias and Variance

An estimator uses sample data to estimate an unknown value or model parameter. Bias describes systematic error: a high-bias model makes assumptions that are too simple. Variance describes sensitivity to the training data: a high-variance model may change a lot when trained on a different sample.

In practice, underfitting is often associated with high bias, while overfitting is often associated with high variance. The aim is not to remove both completely, but to find a reasonable balance.

Maximum Likelihood Estimation (MLE): chooses parameter values that make the observed training data as likely as possible.

Maximum A Posteriori (MAP) is similar to MLE, but it also includes prior belief about the parameters.

Suppose we have data $\mathcal{D} = \{x^{(i)}\}_{i=1}^N$

$$\theta^{\text{MLE}} = \underset{\theta}{\operatorname{argmax}} \prod_{i=1}^N p(\mathbf{x}^{(i)} | \theta) \quad \text{Maximum Likelihood Estimate (MLE)}$$

$$\theta^{\text{MAP}} = \underset{\theta}{\operatorname{argmax}} \prod_{i=1}^N p(\mathbf{x}^{(i)} | \theta) \underbrace{p(\theta)}_{\text{Prior}} \quad \text{Maximum a posteriori (MAP) estimate}$$

Cross-Entropy

Cross-entropy is a loss function commonly used for classification. It compares the true class with the probability predicted by the model. Confident correct predictions receive a small loss, while confident wrong predictions receive a large loss. If $h(x)$ is the prediction output of the classification and y is the actual value, then **Binary cross-entropy**:

$$\text{Cost}(h(x), y) = -y \log(h(x)) - (1 - y) \log(1 - h(x))$$

$$\text{if } y = 0, \text{ Cost}(h(x), y) = -\log(h(x))$$

$$\text{if } y = 1, \text{ Cost}(h(x), y) = -\log(1 - h(x))$$

Scenario and Problem to be Solved

Imagine that you are building a simple AI system that predicts the species of an iris flower from measurements of its sepals and petals. The goal is not only to obtain high accuracy. You also want to understand how the complete machine learning pipeline affects the result.

For this lab, use the Iris dataset available directly in scikit-learn. It contains 150 flower samples, four numerical features, and three target classes. Because the dataset is small and already cleaned, it is suitable for experimenting with training, validation, model selection, and loss.

Setup

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```

iris = load_iris()
X, y = iris.data, iris.target

# First split: keep 20% as a final test set
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Second split: create training and validation sets
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

```

Assignments

Assignment (1): Build the pipeline: Load the Iris dataset and prepare training, validation, and test sets. Write down the number of samples in each set. Explain in two or three sentences why the test set should not be used while choosing a model or hyperparameter.

Assignment (2): Compare a generative and a discriminative model: Train Gaussian Naive Bayes as a generative model and Logistic Regression as a discriminative model. Evaluate both on the validation set. Record the validation accuracy of each model and identify which one performs better.

Assignment (3): Observe model capacity: Train Decision Tree classifiers using `max_depth = 1, 3, 5`, and `None`. For each model, record training accuracy and validation accuracy. Identify one setting that underfits and one setting that appears to overfit. Explain your choice.

Assignment (4): Select a hyperparameter using validation data: Using the results from Assignment (3), choose the `max_depth` that gives the best validation performance. After choosing it, evaluate that model once on the final test set. Report the final test accuracy.

Assignment (5): Bias and variance discussion: Compare the shallow and deep decision trees. In your own words, explain which model is more likely to have high bias and which one is more likely to have high variance. Relate your answer to the training and validation accuracies you observed.

Assignment (6): Cross-entropy: For Logistic Regression, obtain class probabilities using `predict_proba()`. Compute the validation log loss using `sklearn.metrics.log_loss`. Find one confidently correct prediction and one incorrect prediction, and compare how their probabilities contribute to cross-entropy.

Assignment (7): MLE and MAP reflection: Logistic Regression is commonly trained by maximizing likelihood (equivalently, minimizing a related log-loss). Regularization can be viewed as adding a preference for simpler parameter values, which is related to MAP estimation. In four or five sentences, explain the practical difference between MLE and MAP in your own words.

Questions to be answered

1. Why can a model have very high training accuracy but lower validation accuracy?
2. Why is a validation set useful when choosing hyperparameters?
3. How are underfitting, high bias, and low model capacity related?
4. How are overfitting, high variance, and high model capacity related?
5. What extra information does MAP use that MLE does not?
6. Why does cross-entropy heavily penalize a confident but wrong prediction?